example, the Ubuntu image for version 22.04 is called `ubuntu:22.04`. A colon separates the base name and version.

A *container* is a runnable instance of a container image. You can manage the container life cycle from the Docker CLI or Docker Compose: you can start, stop, update, and delete containers. Containers are defined by the image on which they are based and the configuration provided at startup time (for example, environment variables used to customize the container). By default, containers are isolated from each other and the host machine, but you can make them expose services to the outside world through specific ports with a process called *port forwarding* or *port mapping*. Containers can have any name. If you don't specify one, the Docker server will assign a random one, like `bazinga_schrodinger`. To run OCI images as containers, you'll need Docker or any other container runtime compatible with the OCI specifications.

When you want to run a new container, you can use the Docker CLI to interact with the Docker daemon, which checks whether the specified image is already present on the local server. If not, it will find the image on a registry, download it, and then use it to run a container. The workflow, again, is shown in figure 2.7.

Docker on macOS and Windows. How does it work?

In the previous chapter, you learned that containers share the same operating system kernel and rely on Linux features such as namespaces and cgroups. We'll use Docker to run Spring Boot applications in Linux containers, but how is Docker supposed to work on a macOS or Windows machine?